

Tabulation and the 0/1 Knapsack Problem

Programming and Data Structures — Week 9, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement bottom-up tabulation for Fibonacci and explain precisely how it differs from Lecture 1's top-down memoization; state the 0/1 knapsack problem precisely and identify its overlapping subproblems and optimal substructure explicitly, rather than assuming dynamic programming applies; build and read a complete knapsack dynamic programming table by hand on a small concrete instance; and reconstruct the actual set of chosen items from a filled table, not merely the optimal total value.

Outline

Today covers a bottom-up-versus-top-down analogy for two ways of applying dynamic programming; tabulation, rebuilding Fibonacci without any recursion at all; the 0/1 knapsack problem, stated precisely; explicitly identifying its overlapping subproblems and optimal substructure; building its complete dynamic programming table by hand on a small instance; reconstructing the actual chosen items from that filled table; the MTech-track space optimization reducing memory from $O(nW)$ to $O(W)$; an in-class quiz; and a summary.

The bottom-up-versus-top-down analogy

Lecture 1's memoization works top-down: starting from the big question, `fib(n)`, the algorithm recurses down to progressively smaller subproblems, caching each result as the recursion unwinds back upward. Tabulation works in the opposite direction, bottom-up: starting from the smallest subproblems, `fib(0)` and `fib(1)`, and building up to the target `n` directly, using a simple loop with no recursion at all. Both approaches rest on the identical underlying idea — cache every distinct subproblem's answer, computing each exactly once — differing only in the direction the table gets filled.

Tabulation: Fibonacci, rebuilt bottom-up

```
def fib_tabulated(n):
    if n <= 1:
        return n
    table = [0] * (n + 1)
    table[1] = 1
    for i in range(2, n + 1):
        table[i] = table[i - 1] + table[i - 2]
    return table[n]
```

This tabulated version uses no recursion whatsoever — a plain array, directly reusing Week 2’s structure, filled by a straightforward for loop. Each `table[i]` is computed only after `table[i-1]` and `table[i-2]` already exist in the array, which is precisely the ordering guarantee tabulation requires: process subproblems in an order that guarantees every dependency is already resolved. This achieves the same $O(n)$ time as Lecture 1’s memoization, but without that version’s $O(n)$ call-stack depth cost — a genuine space advantage, since no recursive calls are made at all.

The 0/1 knapsack problem, stated precisely

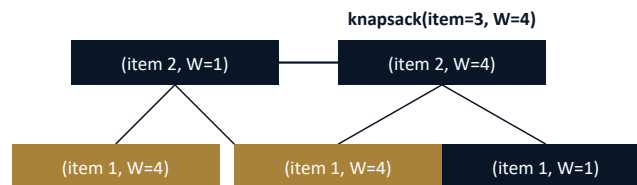
The 0/1 knapsack problem: given a knapsack with weight capacity W , and n available items, each with a weight w_i and a value v_i , choose a subset of items that maximizes total value while keeping total weight within the capacity W . The “0/1” in the name specifically means each item is a binary choice — either fully included or fully excluded — with no ability to take a fractional amount of an item or take the same item more than once, distinguishing it from the “fractional knapsack” variant Week 10’s greedy algorithms lecture covers.

Why naive recursion applies here, and why it is slow

```
def knapsack_naive(weights, values, n, W):
    if n == 0 or W == 0:
        return 0
    if weights[n-1] > W:
        return knapsack_naive(weights, values, n-1, W) # can't include item n-1
    include = values[n-1] + knapsack_naive(weights, values, n-1, W - weights[n-1])
    exclude = knapsack_naive(weights, values, n-1, W)
    return max(include, exclude)
```

This naive recursive formulation considers, for each item, two choices — include it or exclude it — recursively exploring both and taking whichever yields greater total value. Just as naive Fibonacci recomputed the same n repeatedly, this naive knapsack recomputes the identical (n, W) pair repeatedly across different combinations of inclusion and exclusion decisions made for other items — genuine overlapping subproblems, in a real optimization problem rather than a toy mathematical sequence. The cost is $O(2^n)$, exponential in the number of items, exactly as impractical as naive Fibonacci was for even moderately sized inputs.

The picture: recursion tree with genuinely overlapping (n, W) pairs



Both gold boxes are the IDENTICAL subproblem: $(\text{item } 1, W=4)$ — reached via two different decision paths

Naive recursion recomputes it twice, independently. A DP table computes it once, at row "item 1", column $W=4$.

Unlike Fibonacci's single-argument overlap, knapsack subproblems are identified by a *pair*, $(\text{item index, remaining capacity})$ — two different sequences of include/exclude decisions can independently arrive at the identical remaining capacity for the identical next item, producing exactly the same kind of redundant recomputation Lecture 1 demonstrated, now in a two-dimensional subproblem space.

Optimal substructure, verified explicitly

Verifying optimal substructure explicitly, rather than assuming it: if item n is excluded from the optimal solution, then the remaining items chosen must themselves form an optimal solution to the smaller subproblem of items 1 through $n - 1$ with the same capacity W — if a better selection existed for that subproblem, substituting it would improve the overall solution, contradicting optimality. If item n is included, the remaining chosen items must form an optimal solution to items 1 through $n - 1$ with reduced capacity $W - w_n$, by the identical argument. The overall optimum is simply the better of these two cases — no other combination of decisions needs to be considered, which is exactly what "optimal substructure" means in concrete terms.

Building the DP table, completely, by hand

Building the complete table for 3 items with weights $[1, 3, 4]$ and values $[15, 20, 30]$, capacity $W = 4$: row “0 items” is entirely 0, the base case. Row “item 1” reflects that a weight-1 item worth 15 fits at every capacity $W \geq 1$. Row “items 1-2” shows the weight-3, value-20 item improving the result at $W = 3$ (taking it alone beats taking only item 1) and at $W = 4$ (taking both items 1 and 2, weight $1 + 3 = 4$, value $15 + 20 = 35$, exceeds either alone). Row “items 1-3” shows the weight-4 item does not improve the final answer at any capacity here, since including it (value 30, using all 4 capacity) is worse than the 35 already achieved by items 1 and 2 together.

Filling one cell, traced in complete detail

Filling table[items 1-2][W=4]:

Item 2: weight=3, value=20. Current $W=4 \geq \text{weight } 3$, so item 2 CAN fit.

Option A (exclude item 2): table[item 1][W=4] = 15

Option B (include item 2): value 20 + table[item 1][W=4-3=1] = 20 + 15 = 35

$\max(15, 35) = 35 \rightarrow \text{table}[\text{items 1-2}][W=4] = 35$

Tracing the computation of a single cell in complete detail makes the general recurrence concrete: filling the entry for “items 1-2, capacity 4” checks whether item 2 (weight 3) fits within capacity 4, which it does. Excluding item 2 gives whatever the previous row already achieved at the same capacity, 15. Including item 2 gives its value, 20, plus whatever the previous row achieved at the *reduced* capacity, $4 - 3 = 1$, which was 15, giving $20 + 15 = 35$. The cell takes the maximum of these two options, 35 — this exact two-option comparison, generalized, is the entire knapsack recurrence, applied to every cell in the table.

Reconstructing the chosen items, not just the value

```
def reconstruct(table, weights, values, n, W):
    chosen = []
    for i in range(n, 0, -1):
        if table[i][W] != table[i-1][W]:
            chosen.append(i - 1)
            W -= weights[i - 1]
    return chosen
```

value CHANGED → item i was included
record it (0-indexed)
reduce remaining capacity accordingly

The filled table directly gives the optimal total value, but not, on its own, which specific items achieve it. Reconstructing the chosen items walks the table backward from the final cell: if the value at row i , capacity W differs from the value at row $i - 1$ at the same capacity, item i must have been included

to produce that improvement, and the remaining capacity is reduced accordingly before continuing to check row $i - 1$. This reconstruction is possible specifically because the entire table was preserved, a natural byproduct of tabulation's array-based approach.

Tracing reconstruction on this lecture's example

```
table[3][4]=35, table[2][4]=35 → SAME → item 3 excluded
table[2][4]=35, table[1][4]=15 → DIFFERENT → item 2 INCLUDED.  $W = 4 - 3 = 1$ 
table[1][1]=15, table[0][1]=0 → DIFFERENT → item 1 INCLUDED.  $W = 1 - 1 = 0$ 
Chosen items: item 2, item 1 → total value  $20 + 15 = 35$ , total weight  $3 + 1 = 4$ 
```

Tracing reconstruction on the filled table: comparing row 3 against row 2 at $W = 4$ shows no change (both 35), confirming item 3 was correctly excluded. Comparing row 2 against row 1 at $W = 4$ shows a change (35 versus 15), confirming item 2 was included, reducing the tracked capacity to $4 - 3 = 1$. Comparing row 1 against row 0 at the new $W = 1$ shows another change (15 versus 0), confirming item 1 was included. The reconstructed selection — items 1 and 2 — has total value $15 + 20 = 35$, matching the table exactly, and total weight $1 + 3 = 4$, exactly at capacity.

MTech addition — space optimization: $O(nW)$ to $O(W)$

```
def knapsack_optimized(weights, values, n, W):
    table = [0] * (W + 1)
    for i in range(n):
        for w in range(W, weights[i] - 1, -1):  # iterate BACKWARD
            table[w] = max(table[w], values[i] + table[w - weights[i]])
    return table[W]
```

Observing that each row of the full table only ever depends on the row immediately above it, all earlier rows can be discarded, collapsing memory from $O(nW)$ down to $O(W)$ — a single one-dimensional array reused across all n items. The backward iteration over w within each item's pass is essential, not a stylistic choice: iterating forward would allow an item to be “picked up” more than once within the same pass, silently turning this into the unbounded-knapsack variant rather than 0/1 knapsack. The real trade-off: this space optimization makes item reconstruction, traced earlier this lecture, impossible, since the intermediate rows needed for that backward walk no longer exist — a direct, concrete instance of the general time/space/information trade-offs this course has surfaced repeatedly since Week 2.

Cost of the full DP solution

The full dynamic programming table has $(n + 1) \times (W + 1)$ cells, and each cell costs $O(1)$ to fill, since it only compares two already-computed values. Total cost is therefore $O(nW)$ — polynomial in both the number of items and the capacity, compared to naive recursion's $O(2^n)$, exponential in the number of items alone. For a concrete comparison, 30 items with capacity 100 requires roughly 3,000 operations under dynamic programming, versus over one billion under naive recursion — this is $O(nW)$ replacing $O(2^n)$ for exactly the same problem, the same category of transformation Lecture 1 demonstrated for Fibonacci, now applied to a genuine optimization problem with real decision-making at each step.

Exercise

As an exercise, build the complete dynamic programming table by hand for 4 items with weights $[2, 3, 4, 5]$ and values $[3, 4, 5, 6]$, capacity $W = 5$, and use this lecture's reconstruction technique to identify the actual chosen items achieving the optimal value. Then implement `knapsack_optimized` and confirm it returns the identical optimal value (though not the chosen items themselves) as the full-table version on the same test case.

“Pseudo-polynomial”: a term worth knowing precisely

It is worth knowing the precise term for this algorithm's complexity class: $O(nW)$ looks polynomial at a glance, but W is a numeric value — the capacity — not a measure of the input's actual size in the usual computational sense (the number of bits needed to write W down is only $O(\log W)$). If W happens to be astronomically large, say 10^{18} , while the number of items n remains small, this “polynomial” algorithm is still impossibly slow in practice, despite the polynomial-looking formula. This is called pseudo-polynomial time: polynomial in the numeric values involved, but not in the input's actual encoded size — a distinction that matters in advanced algorithm analysis and is worth recognizing precisely, even though a full treatment is beyond this course's scope.

Quiz — identify the recurrence

Given an item i whose weight w_i exceeds the current capacity W being considered, determine what `table[i][W]` must equal, before checking the answer.

Quiz — answer

When an item's weight exceeds the current capacity, it structurally cannot be included at all — there is no “include” option to compare against, unlike the general case. `table[i][W]` simply equals `table[i-1][W]`, carrying forward whatever value the previous row already achieved at that same capacity, unchanged. This is a base-case-like simplification of the general two-option recurrence, applying whenever the “include” option is not physically available.

A closing check: verifying the naive recursion is genuinely correct first

A closing discipline worth stating explicitly: before adding memoization or tabulation to any recursive solution, first confirm the naive recursive version itself is correct on small, hand-checkable test cases. Memoization and tabulation both preserve whatever logic the underlying recurrence encodes — they change performance, not correctness. Optimizing an already-incorrect recursion simply produces a fast, confidently wrong answer, which can be considerably harder to debug than a slow, wrong one, since the wrong answer now returns instantly. This ordering — verify correctness first, then apply performance techniques — applies to every dynamic programming problem in this unit and beyond, not merely knapsack.

Summary

Tabulation rebuilds Lecture 1's memoization approach bottom-up, using a plain array and a simple loop instead of recursion, achieving the same $O(n)$ time without the recursive call stack's overhead. The 0/1 knapsack problem genuinely exhibits both overlapping subproblems and optimal substructure, verified explicitly in this lecture rather than merely assumed, making it dynamic programming's first genuine optimization example in this course. The DP table's core recurrence, at every cell, is a simple two-option comparison — include the current item, or exclude it — and reconstructing the actual chosen items, not merely the optimal value, requires walking the completed table backward, a capability lost if memory is optimized down to $O(W)$. Next lecture introduces greedy algorithms, a different and often faster technique for optimization problems, correct only when a considerably stronger structural property than optimal substructure alone actually holds.